Data Structures-1 Laboratory (DS-1 L)

S.Y. B.Tech CSE, Semester III, AY 2026-27

Experiment Number: 05

Circular Linked List Operations – Polynomial Addition

1. C Program Code

Menu-driven program following the pseudo code of Experiment 5: create() with a head node, insert() that keeps terms in descending order of exponent and combines like terms, display(), and add_poly(). Each node stores a coefficient, an exponent and a pointer; the last node points back to the head node.

```
#include <stdio.h>
#include <stdlib.h>

/* Node of the circular linked list: one polynomial term */
typedef struct node {
    int coef;               /* coefficient */
    int exp;                /* exponent    */
    struct node *next;
} pnode;

/* ---------- allocate a new node ---------- */
pnode *getnode(void) {
    pnode *n = (pnode *)malloc(sizeof(pnode));
    if (n == NULL) {
        printf("Memory allocation failed!\n");
        exit(1);
    }
    n->coef = 0;
    n->exp = 0;
    n->next = n;
    return n;
}

/* ---------- insert a term in descending order of exponent ---------- */
void insert(pnode *head, pnode *newnode) {
    pnode *p, *prev;

    if (newnode->coef == 0) {          /* never store zero terms */
        free(newnode);
        return;
    }
    p = head->next;
    prev = head;
    while (p != head && p->exp > newnode->exp) {   /* find the position for insertion */
        prev = p;
        p = p->next;
    }
    if (p != head && p->exp == newnode->exp) {     /* exponent already exists */
        p->coef = p->coef + newnode->coef;         /* add coefficients */
        if (p->coef == 0) {                        /* if coefficient becomes zero */
            prev->next = p->next;                  /* pointer adjustment before freeing
*/
            p->next = NULL;
            free(p);
        }
        free(newnode);
    } else {                                       /* insert in between or at end */
        newnode->next = p;
        prev->next = newnode;
    }
}

/* ---------- create polynomial ---------- */
pnode *create(void) {
    pnode *head, *newnode;
```

```
    int i, n;

    head = getnode();
    head->next = head;
    head->exp = 999;                    /* temporary sentinel while building */
    printf("Enter number of terms in polynomial: ");
    scanf("%d", &n);
    for (i = 1; i <= n; i++) {
        newnode = getnode();
        printf("Term %d - Coefficient: ", i);
        scanf("%d", &newnode->coef);
        printf("Term %d - Exponent   : ", i);
        scanf("%d", &newnode->exp);
        if (newnode->exp < 0) {
            printf("Negative exponent not allowed, term skipped.\n");
            free(newnode);
            continue;
        }
        insert(head, newnode);
    }
    head->exp = -1;                     /* head marks the end of the polynomial */
    return head;
}

/* ---------- display polynomial in readable form ---------- */
void display(pnode *head) {
    pnode *p = head->next;
    int first = 1, c;

    if (p == head) {
        printf("0\n");
        return;
    }
    while (p != head) {
        c = p->coef;
        if (first) {
            if (c < 0) printf("-");
        } else {
            printf(c < 0 ? " - " : " + ");
        }
        if (c < 0) c = -c;
        if (p->exp == 0) {
            printf("%d", c);
        } else {
            if (c != 1) printf("%d", c);
            printf("x");
            if (p->exp != 1) printf("^%d", p->exp);
        }
        first = 0;
        p = p->next;
    }
    printf("\n");
}

/* ---------- add two polynomials: result = p1 + p2 ---------- */
pnode *add_poly(pnode *p1, pnode *p2) {
    pnode *t1 = p1->next, *t2 = p2->next;
    pnode *h = getnode(), *t3 = h, *curr;
    int coef, exp;

    h->exp = -1;
    h->next = h;
    while (t1->exp != -1 || t2->exp != -1) {
        if (t1->exp == t2->exp) {
            coef = t1->coef + t2->coef;
            exp = t1->exp;
            t1 = t1->next;
            t2 = t2->next;
        } else if (t1->exp > t2->exp) {
            coef = t1->coef;
            exp = t1->exp;
```

```
            t1 = t1->next;
        } else {
            coef = t2->coef;
            exp = t2->exp;
            t2 = t2->next;
        }
        if (coef != 0) {               /* skip terms whose coefficient is zero */
            curr = getnode();
            curr->exp = exp;
            curr->coef = coef;
            t3->next = curr;
            curr->next = h;
            t3 = curr;
        }
    }
    return h;
}

/* ---------- free all nodes (including head) ---------- */
void destroy(pnode *head) {
    pnode *p = head->next, *t;
    while (p != head) {
        t = p;
        p = p->next;
        free(t);
    }
    free(head);
}

/* ---------- verify that the last node points back to the first node ---------- */
void verify(pnode *head, const char *name) {
    pnode *p = head;
    int count = 0;
    while (p->next != head) {
        p = p->next;
        count++;
    }
    if (count == 0)
        printf("%s: empty polynomial, head points to itself (circular).\n", name);
    else
        printf("%s: %d term(s); last node points back to the first node (head): %s\n",
               name, count, (p->next == head) ? "YES" : "NO");
}

int main(void) {
    pnode *p1 = getnode(), *p2 = getnode(), *res;
    int choice;

    p1->exp = -1;
    p2->exp = -1;
    do {
        printf("\n===== Polynomial Operations (Circular Linked List) =====\n");
        printf("1. Create Polynomial 1\n2. Create Polynomial 2\n3. Display
Polynomials\n");
        printf("4. Add Polynomials\n5. Verify Circular Link\n0. Exit\n");
        printf("Enter your choice: ");
        scanf("%d", &choice);

        switch (choice) {
            case 1: destroy(p1); p1 = create(); break;
            case 2: destroy(p2); p2 = create(); break;
            case 3:
                printf("P1(x) = "); display(p1);
                printf("P2(x) = "); display(p2);
                break;
            case 4:
                res = add_poly(p1, p2);
                printf("P1(x) = "); display(p1);
                printf("P2(x) = "); display(p2);
                printf("Sum   = "); display(res);
                destroy(res);
```

```
                break;
            case 5: verify(p1, "P1"); verify(p2, "P2"); break;
            case 0: printf("Exiting...\n"); break;
            default: printf("Invalid choice!\n");
        }
    } while (choice != 0);

    destroy(p1);
    destroy(p2);
    return 0;
}
```

2. Output

Compiled with gcc -Wall -Wextra -o sll poly.c (no warnings) and executed on Linux. The screenshots below show the program output for the test conditions listed in the lab write-up.

Test 1 – Create and display: empty polynomial, multiple terms entered out of order, single term, duplicate exponents combined, circular-link check

Test 1: Create / Display / Verify circular link (part 1)

```
student@linux:~/ds1$ ./poly
===== Polynomial Operations (Circular Linked List) =====
1. Create Polynomial 1
2. Create Polynomial 2
3. Display Polynomials
4. Add Polynomials
5. Verify Circular Link
0. Exit
Enter your choice: 3
P1(x) = 0
P2(x) = 0

===== Polynomial Operations (Circular Linked List) =====
1. Create Polynomial 1
2. Create Polynomial 2
3. Display Polynomials
4. Add Polynomials
5. Verify Circular Link
0. Exit
Enter your choice: 1
Enter number of terms in polynomial: 4
Term 1 - Coefficient: 2
Term 1 - Exponent   : 2
Term 2 - Coefficient: 5
Term 2 - Exponent   : 3
Term 3 - Coefficient: 7
Term 3 - Exponent   : 0
Term 4 - Coefficient: -4
Term 4 - Exponent   : 1

===== Polynomial Operations (Circular Linked List) =====
1. Create Polynomial 1
2. Create Polynomial 2
3. Display Polynomials
4. Add Polynomials
5. Verify Circular Link
0. Exit
Enter your choice: 2
Enter number of terms in polynomial: 1
Term 1 - Coefficient: 6
Term 1 - Exponent   : 4
```

Figure A1: Program output (part 1 of 3)

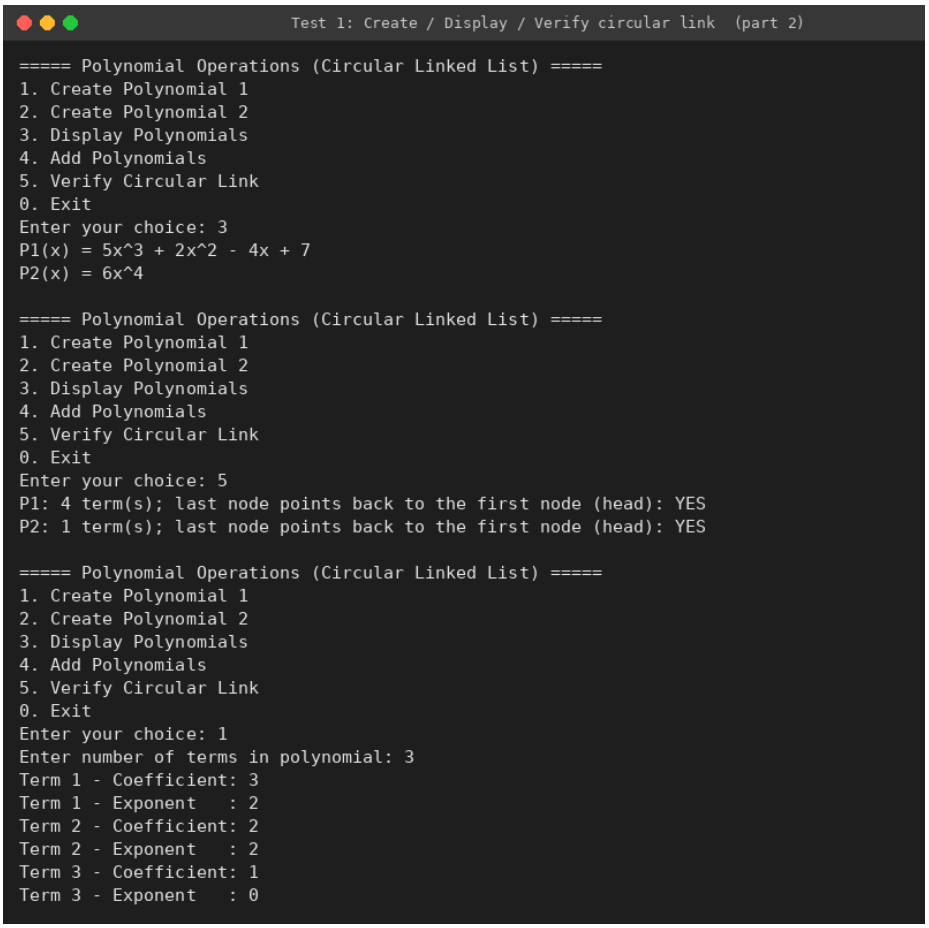


Figure A2: Program output (part 2 of 3)

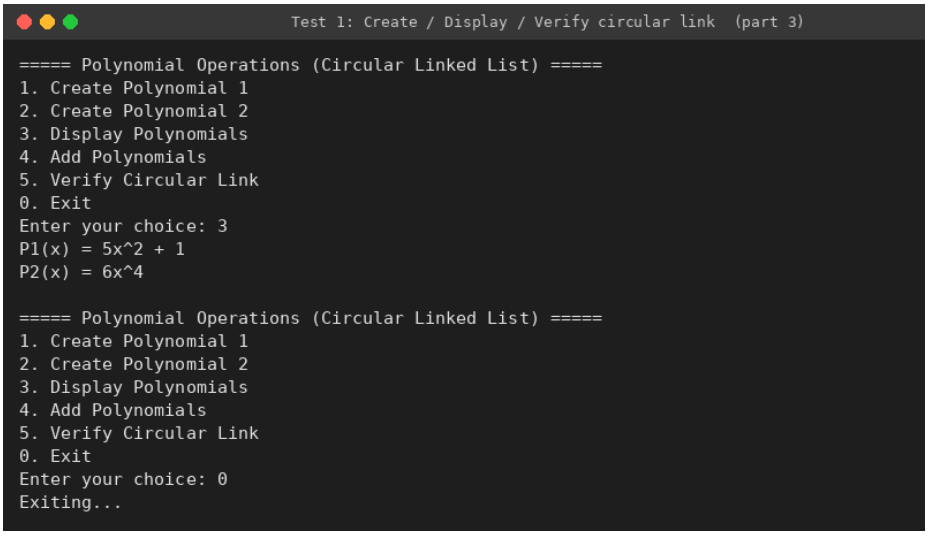


Figure A3: Program output (part 3 of 3)

Test 2 – Add two polynomials having common exponents (a zero-sum term is dropped)



Figure B1: Program output (part 1 of 2)

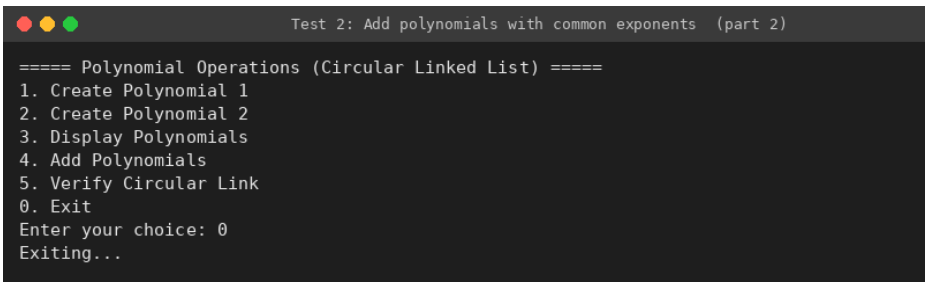


Figure B2: Program output (part 2 of 2)

Test 3 – Add polynomials with no common exponents, with negative coefficients, and with an empty polynomial

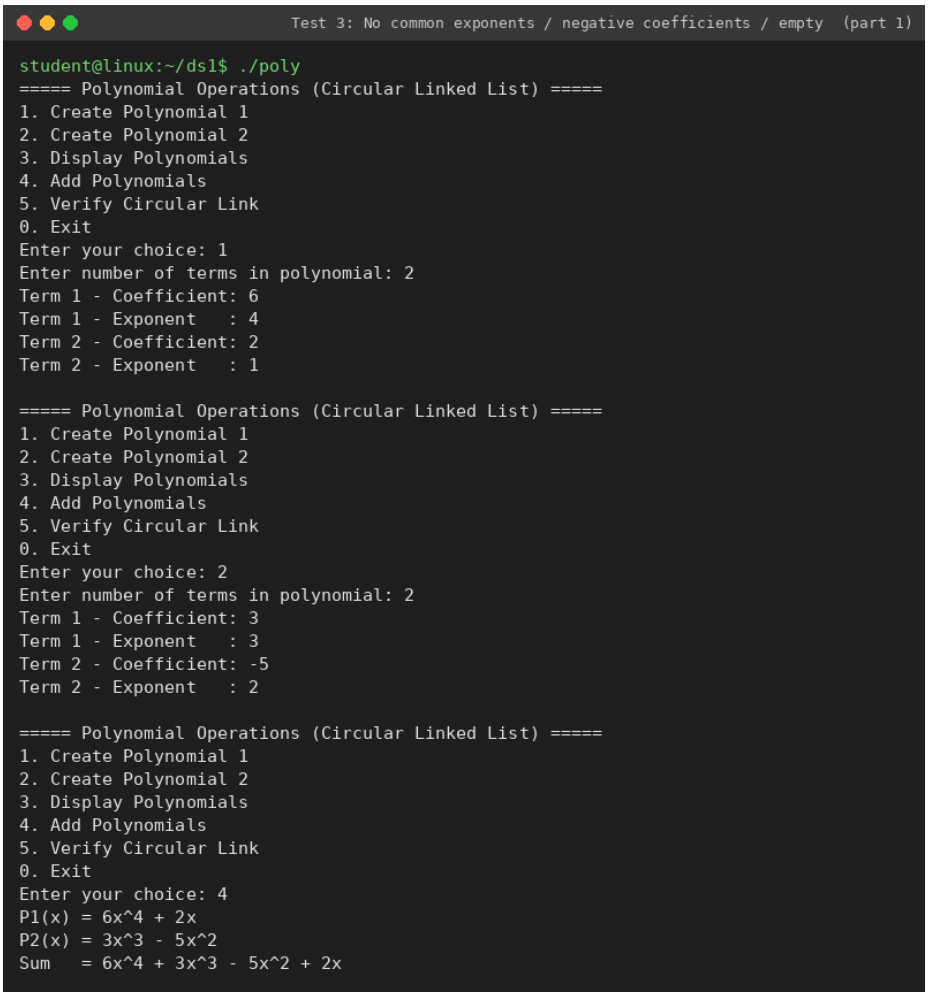


Figure C1: Program output (part 1 of 3)

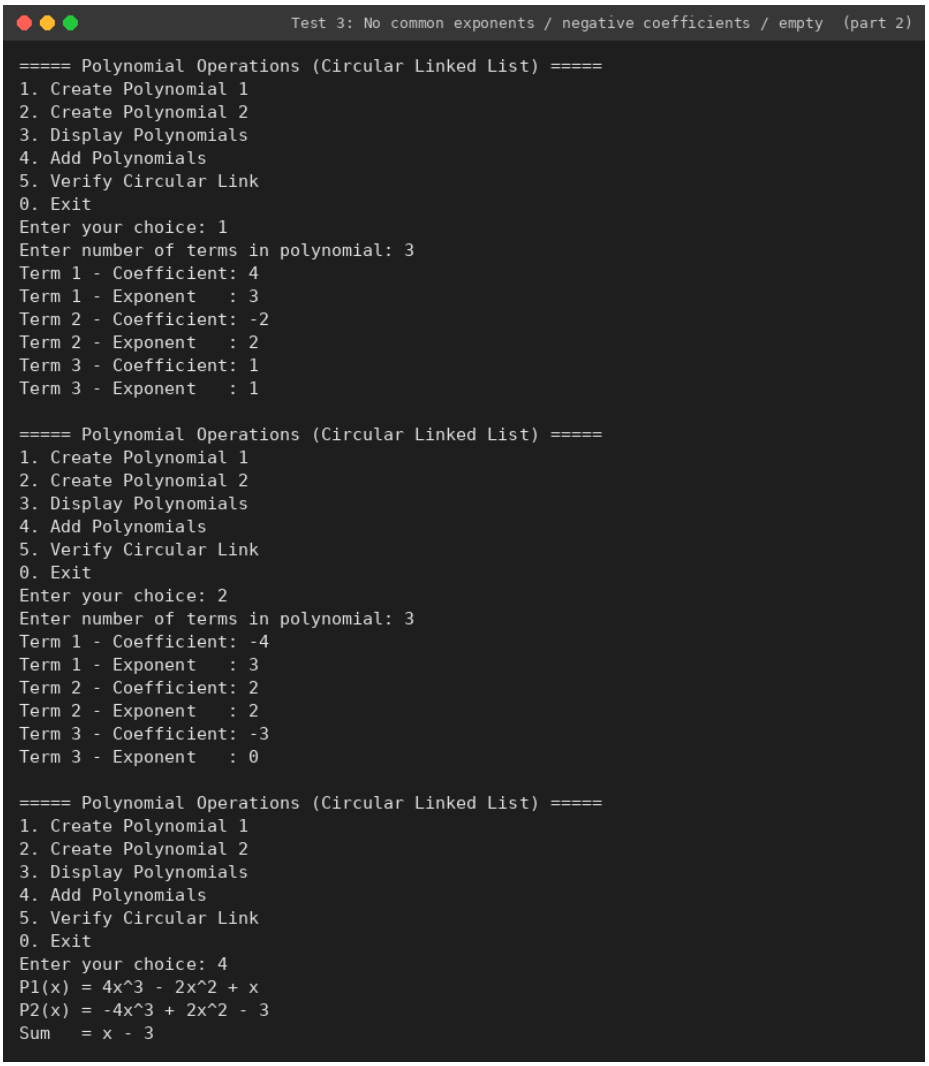


Figure C2: Program output (part 2 of 3)

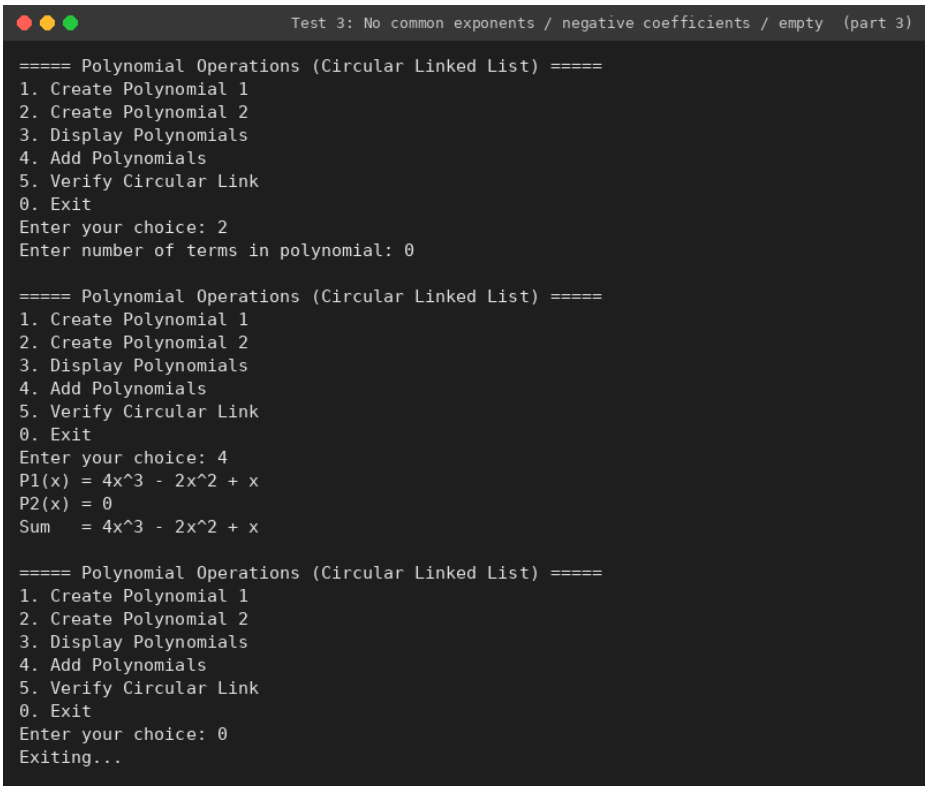


Figure C3: Program output (part 3 of 3)